

Software Engineer

Product Engineer

Frontend Developer

가천대학교 컴퓨터공학과 GPA 3.99 / 4.5

PROFILE

React·TypeScript로 WebView 거래 제품과 BFF, 블록 스캐너, 사내 개발 도구를 만들어왔습니다. 구현에 앞서 데이터 흐름과 시스템 경계를 정리하고, 성능 개선은 측정값을 기준으로 판단합니다. 제품 문제를 해결하는 데 필요하면 NestJS·FastAPI 등 백엔드 영역까지 직접 다룹니다.

EXPERIENCE

2024.09 - 현재

Frontend Developer

2년

파이랩테크놀로지

Frontend Developer

WebView 거래 제품과 BFF, 블록 스캐너 성능 개선, 사내 AI Agent 런타임 개발을 맡고 있습니다.

TypeScript

React

Next.js

TailwindCSS

TanStack Query

Jotai

NestJS

wagmi · viem

Chakra UI

Vitest

Playwright

Sentry

- 블록 스캐너 주요 목록에 단계적 렌더링과 WebSocket 배치를 적용해 로컬 Lighthouse 기준 LCP 평균 **45%**, TBT 평균 **74%** 단축
- BiFi·Swap·BTCFi 거래 화면을 Mobile App·Chrome Extension에서 공통 WebView로 재사용하도록 BFF 계약과 postMessage 연동 구조 설계
- 여러 RPC·사내 API 호출을 BFF 집계 API로 묶어 자산 수치가 응답 순서에 따라 달라지던 문제 해결
- 저장된 isNative 값 오류로 native 코인 송금이 ERC-20 호출로 만들어질 수 있던 문제를 주소 비교 방식으로 수정하고 회귀 테스트 추가

2021.07 - 2024.09

3년 2개월

Full-stack Engineer

→ Frontend Developer

→ Frontend Part Leader

스마트마인드

Frontend Developer → Frontend Part Leader

AI 데이터 분석 플랫폼 Workspace의 SQL Editor·Query Viewer 모듈을 개발했으며, pnpm·Turborepo 모노레포 전환과 Microfrontend 설계를 주도했습니다. 후반에는 프론트엔드 파트 리더로 팀 운영을 담당했습니다.

TypeScript

React

Next.js

Vite

Monaco Editor

ANTLR

pnpm · Turborepo

Playwright

FastAPI

Docker Compose

- 번들 분리·dynamic import로 Workspace 초기 로드 범위를 줄여 당시 측정 기준 First Load 최대 **40% 단축**
- pnpm·Turborepo 모노레포와 빌드 캐시를 도입해 당시 측정 기준 앱별 빌드 시간을 **7분 → 1분**으로 단축
- AI 팀이 정의한 ANTLR 문법을 Monaco Editor에 연결해 SQL 구문 강조·오류 진단·키워드 자동완성 구현
- Microfrontend 설계·구현 — iframe+postMessage(Lab·Main 간 통신), module federation(Query Manager·File Manager)
- 비정형 데이터 Query Viewer 성능 개선 — 서버 페이지네이션·가상 스크롤·미디어 lazy loading으로 대용량 결과 렌더링 안정화
- FastAPI·Docker Compose로 AI 예측 모델 서빙 인프라 구성 및 데이터 수집·전처리 파이프라인 직접 개발
- 2023년부터 본인 포함 2~3명 프론트엔드 파트의 스크럼 운영·업무 배분·우선순위 결정 담당

01 /

블록 스캐너(Explorer) 성능 최적화

Route	LCP (before → after)	TBT (before → after)	개선
/blocks	3.7s 1.9s	1,701ms 270ms	LCP ↓49% · TBT ↓73%
/txs	3.7s 2.2s	1,330ms 200ms	LCP ↓41% · TBT ↓85%
/tokens	3.3s 1.8s	510ms 190ms	LCP ↓45% · TBT ↓63%

- 최대 75개 row를 노출하는 테이블에서 가상화 적용 시 레이아웃 공백 문제가 있어 롤백하고, 초기 row 수를 제한한 단계적 렌더링으로 전환
- WebSocket 30초 배치 처리로 실시간 업데이트에 따른 반복 렌더링 축소

02 /

DApp 데이터 조회 구조 최적화

- Gas Top-up — SDK가 환율 계산을 위해 반복하던 asset 조회 의존을 분리하고, GTU API가 이미 제공하는 vault.network asset의 중복 조회를 제거해 최초 호출 25~46회→17회, 주기 호출 분당 25회→1~19회로 축소
- BiFi WebView — bridge pair 최초 1회+pair별 N회 조회를 outbound·inbound 각 1회로 재구성하고 React Query 캐시를 적용해 반복 조회 제거
- BiFi WebView — balance 조회를 background dispatcher 경유에서 클라이언트 단일 호출로 바꾸고 거래 성공 직후 즉시 갱신

03 /

모바일 WebView 거래 제품·안정성 개선

- Pockie miniDApp — BiFi·Swap·BTCFi 화면은 공통 WebView로 배포하고, 지갑 서명·전송만 Mobile App·Chrome Extension에 위임하도록 구조화
- Pockie miniDApp — iOS·Android·Extension과 앱 버전별 feature flag를 BFF config v2에 구현해 클라이언트 재배포 없이 기능 노출 제어
- BTCFi Partners — 파트너사 iOS·Android 지갑에 WebView 거래 제품을 연동하고, postMessage·widget route로 일반 웹과 상품 로직 공유
- 거래 안정성 — native 코인 오판정으로 잘못된 ERC-20 호출이 생성될 수 있던 문제를 주소 기반 재판정으로 수정

04 /

사내 AI Agent 런타임 개선

- 요청마다 프로세스와 모든 MCP를 띄우던 구조를 Codex App Server 세션 기반으로 전환
- 도구의 외부 영향·되돌릴 수 있는지를 판단해 필요한 MCP와 권한만 열고, 취소하기 어려운 작업만 사용자 승인을 받도록 실행 정책 구성
- 자동 코드 리뷰에 confidence·CI 상태 검증을 추가하고 최종 merge는 사람이 확인하도록 운영

05 /

CERTIFICATIONS & ACTIVITIES

정보처리기사 2019
AWS Certified Solutions Architect – Associate (2026)
OSSCA — TS Handbook 한글화 (2020), githru-vscode-ext (2023)
SmileGate Membership AI 1기 (2021)
AUSG (AWSKRUG University Student Group) 1기 (2019)

LANGUAGES

영어 — 기초 비즈니스 (Opic IM1)
일본어 — 기초 비즈니스 (JLPT N3)